M S GOWRISH SHANKAR REDDY

Reg No: 192424270

DSA0304 Natural Language Processing

Activity 1: Backoff

Backoff idea: a trigram model is the most specific model, but many trigrams never appear in the training text, so their probability would be 0. Backoff avoids this by trusting the highest-order n-gram that was actually seen, and falling back to a shorter context only when the longer one is missing. Each time we back off one level, the score is multiplied by the penalty, because a shorter context is less specific and therefore less trustworthy.

Penalty used: 0.4 (the question states 0.4). Rule applied to a phrase w1 w2 w3:

Level 1 (trigram): if count(w1 w2 w3) > 0, score = count(w1 w2 w3) / count(w1 w2). No penalty.

Level 2 (bigram): if the trigram was not seen but count(w2 w3) > 0, score = 0.4 x count(w2 w3) / count(w2).

Level 3 (unigram): if the bigram was not seen either, score = 0.4 x 0.4 x count(w3) / N = 0.16 x P(w3), where N is the total number of words.

Answer format for each phrase: check the trigram, then the bigram, then the unigram, stop at the first level that was seen, and report the score together with that level.

a) "the cat sat"

Step 1: look up the trigram "the cat sat" in the table. If it was seen, the score is count(the cat sat) / count(the cat), with no penalty, and the level used is the trigram.

Step 2: this phrase is expected to be found at the trigram level, because P(sat | the cat) is also the value used in Activity 2. So the answer is: Score = P(sat | the cat), Level = trigram, penalty not applied.

b) "dog ate fish"

Step 1: look up the trigram "dog ate fish". If it was not seen, no trigram score is available, so we back off.

Step 2: look up the bigram "ate fish". If it was seen, the score is 0.4 x count(ate fish) / count(ate). The level used is the bigram and the penalty is applied once.

Step 3: only if "ate fish" was also unseen would we go to the unigram level.

c) "dog sat fish"

Step 1: look up the trigram "dog sat fish". It is a different word order from the phrase in (b), so it is unlikely to have been seen.

Step 2: look up the bigram "sat fish". If this was also not seen, the bigram level fails as well.

Step 3: back off to the unigram. Score = 0.4 x 0.4 x P(fish) = 0.16 x P(fish), where P(fish) = count(fish) / N. The level used is the unigram and the penalty is applied twice.

Comparison: the deeper the model has to back off, the more the score is reduced, so the order of the scores is (a) highest, then (b), then (c) lowest, which matches how natural each phrase is in the training text.

Activity 2: Deleted interpolation

Interpolation idea: instead of choosing one n-gram level, deleted interpolation mixes the trigram, bigram and unigram estimates together. The weights must be non-negative and add up to 1:

P(w3 | w1 w2) = l3 x P(w3 | w1 w2) + l2 x P(w3 | w2) + l1 x P(w3), with l1 + l2 + l3 = 1.

The weights are not guessed. They are tuned on held-out data (text that was deleted from the training set): the best set of lambdas is the one that gives the held-out text the highest probability.

Part A: P(sat | the cat) with l1 = 0.1, l2 = 0.3, l3 = 0.6

Check: 0.1 + 0.3 + 0.6 = 1.0, so the weights are valid.

Formula: P(sat | the cat) = 0.6 x P(sat | the cat) + 0.3 x P(sat | cat) + 0.1 x P(sat).

Trigram term: 0.6 x count(the cat sat) / count(the cat)

Bigram term: 0.3 x count(cat sat) / count(cat)

Unigram term: 0.1 x count(sat) / N

Add the three terms. The highest weight (0.6) goes to the trigram, so the most specific evidence counts the most, while the bigram and unigram terms keep the result above zero even for unseen contexts.

Part B: held-out sentence "the dog ate fish"

The sentence contains two trigrams: "the dog ate" and "dog ate fish". The score of the sentence is the sum of the natural logs of their interpolated probabilities (summing logs is the same as multiplying the probabilities).

Step 1: for each trigram, find its three component probabilities.

"the dog ate": P(ate | the dog), P(ate | dog), P(ate)

"dog ate fish": P(fish | dog ate), P(fish | ate), P(fish)

Step 2: for each set of lambdas, compute the interpolated probability of each trigram.

Set 1, l = (0.1, 0.3, 0.6): P = 0.6 x P_trigram + 0.3 x P_bigram + 0.1 x P_unigram

Set 2, l = (0.3, 0.6, 0.1): P = 0.1 x P_trigram + 0.6 x P_bigram + 0.3 x P_unigram

Step 3: take ln of each of the two probabilities and add them, giving ln P1 + ln P2 for Set 1 and for Set 2. Both sums are negative, because the probabilities are less than 1.

Step 4: choose the set with the larger sum, that is, the one closer to zero.

Why: deleted interpolation picks the lambdas that maximise the likelihood of the held-out data. A larger sum of ln probabilities means the model assigns the held-out sentence a higher probability, so it predicts unseen text better. Because held-out trigrams such as "the dog ate" are often missing from the training data, a set that puts more weight on the bigram and unigram terms can win, whereas a set that leans on the trigram term does better when trigrams are reliable. The sums computed from the table decide which case applies.

Activity 3: Entropy and perplexity

Definitions: entropy measures the average uncertainty (in bits) when predicting an outcome, H = -sum p log2 p. Perplexity = 2^H is the effective number of equally likely choices the model is choosing between. Higher entropy means more uncertainty and a harder prediction.

Part A: weather (Sunny 0.5, Cloudy 0.25, Rainy 0.25)

Entropy:

H = -[0.5 log2(0.5) + 0.25 log2(0.25) + 0.25 log2(0.25)]

log2(0.5) = -1 and log2(0.25) = -2

H = -[0.5(-1) + 0.25(-2) + 0.25(-2)]

H = 0.5 + 0.5 + 0.5 = 1.5 bits

Perplexity:

Perplexity = 2^H = 2^1.5 = 2.83 (approximately)

Meaning: predicting the weather is as uncertain as choosing between about 2.83 equally likely outcomes. For comparison, three equally likely outcomes would give a perplexity of exactly 3, and the unequal probabilities (Sunny is more likely) make this slightly easier, so it is a little below 3.

Part B: words that come after "the" and after "sat"

Step 1: from the corpus, list every word that follows "the" and its count. Then P(w | the) = count(the w) / count(the), so the probabilities add up to 1.

Step 2: entropy of the next word after "the":

H(next | the) = -sum over w of P(w | the) x log2 P(w | the)

Step 3: perplexity after "the" = 2^H(next | the). This is the effective number of words the model is choosing between at that position.

Step 4: entropy of the next word after "sat". If only one word ever follows "sat", its probability is 1, so:

H = -[1 x log2(1)] = 0 bits, and perplexity = 2^0 = 1

Meaning: after "sat" there is no uncertainty at all, the next word is fully predictable, while after "the" several words are possible, so its entropy is greater than zero. If more than one word follows "sat", use the same sum as in Step 2 with those probabilities.

The word list and probabilities after "the" and "sat" come from the count table of the worksheet.